



Module 25

Introduction to Coverage

cādence®

This training module briefly introduces coverage.

Module Prerequisite

- Experience with verifying digital designs

Module Objectives

In this training module, you

- Identify the coverage types to apply to your project

This training module discusses:

- Coverage fundamentals
- Coverage collection and analysis
- Xcelium™ Integrated Coverage



Your objective is to identify the coverage types to apply to your project.

To help you to attain your objective, this training module discusses:

- Coverage fundamentals;
- Coverage collection and analysis; and
- Xcelium Integrated Coverage.

What Is the Purpose of Coverage?



The **purpose of coverage** is to measure how well the testbench verifies the design.

You use **coverage** to measure how well the testbench verifies the design:

- To identify design areas in which to focus verification efforts.
- To estimate the remaining verification effort.

Verification Metrics	52.61%	380 / 1034 (36.75%)
Types	53.84%	190 / 517 (36.75%)
Instances	51.37%	190 / 517 (36.75%)
dkm_test	51.37%	190 / 517 (36.75%)
dkm_j	51.37%	190 / 517 (36.75%)
coin_counter_j	47.67%	73 / 191 (38.22%)
coin_counter_cgrp_j	56.82%	10 / 17 (58.82%)
COVER_TWO_DIME_OUT_MAKES_EXACT	0%	0 / 1 (0%)
COVER_NICKEL_DIME_OUT_MAKES_EXACT	0%	0 / 1 (0%)
COVER_DIME_OUT_MAKES_EXACT	0%	0 / 1 (0%)
COVER_NICKEL_OUT_MAKES_EXACT	100%	1 / 1 (100%)
COVER_RESET_MAKES_EXACT	100%	1 / 1 (100%)
ASSERT_NEVER_MULTIPLE_ACTION	100%	1 / 1 (100%)
coin_counter_cgrp_j	49.88%	18 / 45 (40%)
cg_coins_j	75.76%	12 / 20 (60%)
nickel_count	27.27%	3 / 11 (27.27%)
dime_count	100%	6 / 6 (100%)
quarter_count	100%	3 / 3 (100%)
can_counter_j	79.29%	27 / 39 (69.23%)
can_counter_prop_j	90%	7 / 8 (87.5%)
COVER_DISP_MAKES_EMPTY	100%	1 / 1 (100%)
COVER_RESET_MAKES_EMPTY	100%	1 / 1 (100%)
ASSERT_NEVER_LOAD_AND_DISPENSE	100%	1 / 1 (100%)
state_machine_j	52.21%	80 / 249 (32.13%)
state_machine_prop_j	57.14%	13 / 22 (59.09%)
COVER_S45_10	0%	0 / 1 (0%)
COVER_S45_25	0%	0 / 1 (0%)
COVER_S40_25	0%	0 / 1 (0%)
COVER_S35_25	0%	0 / 1 (0%)
COVER_S30_25	100%	1 / 1 (100%)
COVER_RESET_MAKES_EMPTY	100%	1 / 1 (100%)
COVER_RESET_MAKES_ZERO	100%	1 / 1 (100%)
ASSERT_NEVER_MULTIPLE_ACTION	100%	1 / 1 (100%)
state_machine_cgrp_j	65%	9 / 15 (60%)
cg_state_j	50%	5 / 10 (50%)
state	50%	5 / 10 (50%)
binds_in_textfile	29.73%	12 / 61 (19.67%)
n/a	n/a	0 / 0 (n/a)

3 © Cadence Design Systems, Inc. All rights reserved.

cadence

Think for a moment, what might the purpose of coverage be?

Coverage is about the *verification environment* – not about the design.

Coverage identifies shortcomings of the verification environment, that result in insufficient verification of the design.

What matters most, is detecting what coverage points are not covered, as these are holes in the verification process, that must be addressed.

By tracking verification progress over time, and comparing it to that of previous projects, you can predict the additional effort required to reach the coverage target.

What Are the Goals of Coverage?

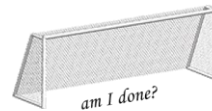
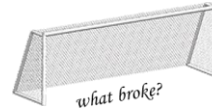


Coverage has three primary **goals**:

- Confirm existing test efficacy.
- Direct further test development.
- Predict test development completion.

Coverage data helps answer these kinds of questions:

- Did my last change break the design?
 - Run a quick regression test
 - You know what tests cover the changed area
- How can I be absolutely sure about the design?
 - Run a complete regression test
 - You know that the tests have high coverage
- What do I test next?
 - Add tests to address areas having lowest coverage
- Is my test development done?
 - Do the coverage metrics meet or exceed the plan?
- When will my test development be done?
 - Track your progress over time



4

© Cadence Design Systems, Inc. All rights reserved.

cadence

The goals of coverage are to confirm efficacy of the existing tests, direct further test development, and help predict test development completion.

Coverage data helps to answer these kinds of questions:

- Did my last change break my design?
 - Coverage can order tests by their effectiveness, so that you can more quickly detect incorrect design operation.
- How confident can I be that my design is correct?
 - Coverage can merge all data to provide one overall coverage metric.
- What do I test next?
 - Coverage reveals the design area least tested that you should next test.
- Is my verification complete?
 - Verification is complete when your desired coverage metric is achieved.
- When will my verification efforts be complete?
 - By tracking the coverage over time and applying experience from past projects, you can reasonably well predict the remaining effort.

What Are the Types of Coverage?

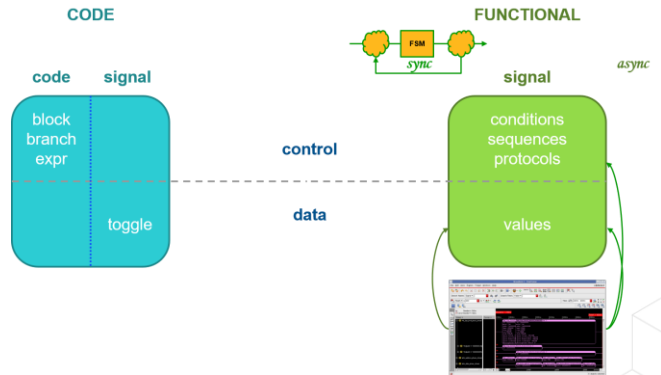


The **types of coverage** are two primary categories:

- Code coverage
- Functional coverage

Coverage types include:

- Code Coverage
 - block, branch, expression, toggle
 - FSM Coverage
 - state, transition, arc
- Functional Coverage
 - data-oriented, control-oriented
 - Transaction Coverage



5 © Cadence Design Systems, Inc. All rights reserved.

cadence

Coverage is in the two primary categories *code coverage* and *functional coverage*, with further sub-categories.

Code coverage can be broadly defined as that coverage that the elaborator can instrument by analyzing the design. This includes code blocks, branches, expressions, and signal states.

State machine coverage is by this definition also a type of code coverage. State machine coverage includes states, transitions between states, and optionally, the condition that caused the transition.

Functional coverage can be broadly defined as that coverage that the user must define. That is further divided as either data-oriented, that is, data values, combinations of data values, and transitions between data values, and control-oriented, that is, sequences of control states.

Transaction coverage is by this definition also a type of functional coverage, as the user must instrument the code to record transaction inception and reception.

What Is Code Coverage?



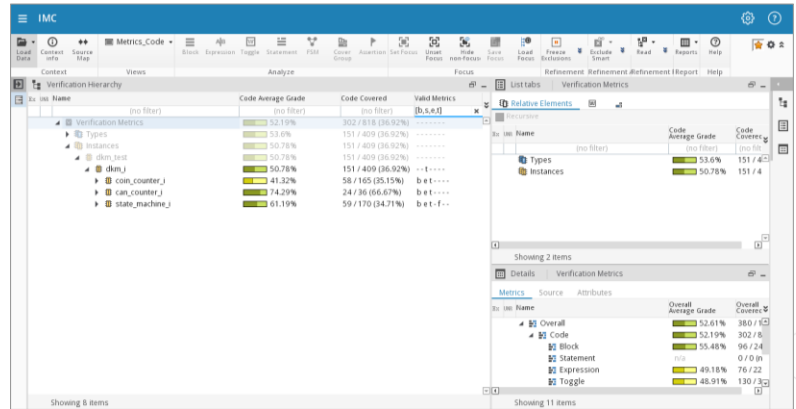
Code coverage is coverage of design code with no knowledge of its *functionality*.

It monitors execution of code blocks, lines, statements, and branches, and can also monitor the state of control expression terms to help explain *why* a code fragment executed (or did not).

Code coverage measures:

- Code block execution
 - Control branch selection
- Expression term states

Abstraction level	Code
Collected	Asynchronously
Domain	Control



6 © Cadence Design Systems, Inc. All rights reserved.

cadence

Code coverage focuses on control statement flow:

- Which blocks executed;
- Which control branches were taken; and
- Which control expression terms controlled the expression value.

Code coverage measures code block execution, including code branches, and expression term states.

Code coverage is collected at the code level of abstraction, asynchronously, in the control domain.

Code coverage should include block coverage. A block is a maximal set of contiguous statements that always execute together. Some tools instead measure line or statement coverage, which is inefficient, and may inappropriately weight the results. Branch coverage is a more precise extension of block coverage. With branch coverage, a block is fully covered only if all its branches are covered.

Code coverage can include expression coverage. Expression coverage detects which operands have controlled the expression, to make it false or true.

Code coverage identifies code that the test suite does not exercise, typically untested code, but also dead code.

Code coverage is relatively easy to count, report and track.

The image is of the code analysis summary page in the Integrated Metrics Center. The overall Verification Metrics are selected. The code average grade at each hierarchy level is the average of the code coverage grades of the next-lower elements. The blocks, expressions, and toggles, are about half covered. Statements were not scored.

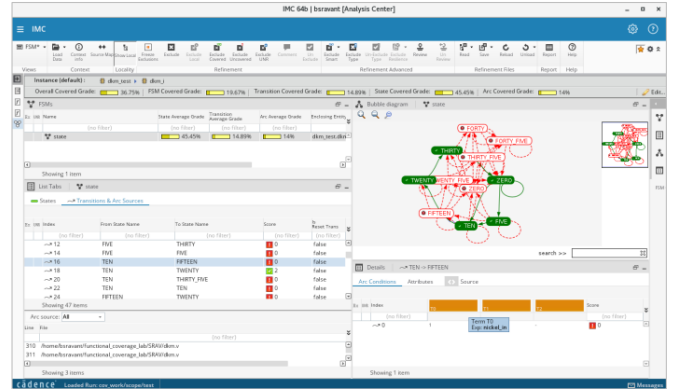
What Is FSM Coverage?



FSM coverage focuses on state machines.

- What states are visited?
 - What state transitions occur?
 - What interactions among FSMs occur?
- It measures states, transitions, and interactions.
 - FSM coverage resembles synchronous control-oriented functional coverage where:
 - FSM states are control signal states
 - FSM state transitions are sequences of control states
 - FSM interactions are cross-products of control states

Abstraction level	Signal
Collected	Synchronously
Domain	Control



FSM coverage focuses on state machines:

- What states were visited;
- What transitions between states occurred; and
- What interactions between state machines occurred.

FSM coverage measures states, transitions, and interactions, and may also utilize expression coverage technology to measure control expressions causing the transitions.

State machine coverage is collected at the signal level of abstraction, synchronously, in the control domain.

FSM coverage is resembles synchronous control-oriented functional coverage, where:

- FSM *states* are control signal states;
- FSM state *transitions* are sequences of control states; and
- FSM *interactions* are cross-products of control states.

The image depicts FSM analysis in the Integrated Metrics Center. We have selected the FSM “state” and the transition TEN to FIFTEEN. The Bubble Diagram highlights the selected state or transition. The Details pane Arc Conditions tab displays the conditions enabling each arc. Here, you see one arc between state TEN and state FIFTEEN, and its condition is that the signal “nickel in” is true.

Note : While FSM coverage and synchronous control-oriented functional coverage both aim to ensure a design functions as intended, they target different aspects. FSM coverage focuses on verifying all the states and transitions of a finite state machine, whereas synchronous control-oriented functional coverage verifies the correctness of the design's control flow under synchronous conditions.

What Is Data-Oriented Functional Coverage?

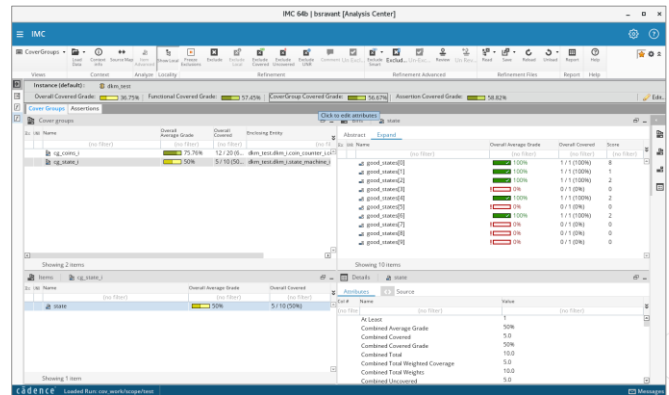


Data-oriented functional coverage is functional coverage that focuses on *data* signals. It measures data signal toggles and values.

Data-oriented functional coverage focuses on data signals:

- Which signals toggled?
- What variable values occurred?
- What combinations of variable values occurred?

Abstraction level	Signal
Collected	Asynchronously/Synchronously
Domain	Data



Data-oriented functional coverage focuses on data signals:

- Which signals toggled;
- What variable values occurred; and
- What combinations of variable values occurred.

Data-oriented functional coverage measures data signal toggles and values.

Data-oriented functional coverage is collected at the signal level of abstraction, asynchronously or synchronously, in the data domain.

You can relatively easily measure and interpret data-oriented functional coverage. You can make it more complex, if you measure large value ranges and cross-products, and specify values and cross products that are illegal or to ignore.

The image is of covergroup analysis in the Integrated Metrics Center. We have selected the “state” covergroup instance, and in it, the “state” coverpoint item. Here, you see some good state values that never occurred. For an FSM that *cannot* be extracted, covergroup coverage can provide state and transition coverage. For an FSM that *can* be extracted, FSM coverage can cover reset and hold transitions, and arcs.

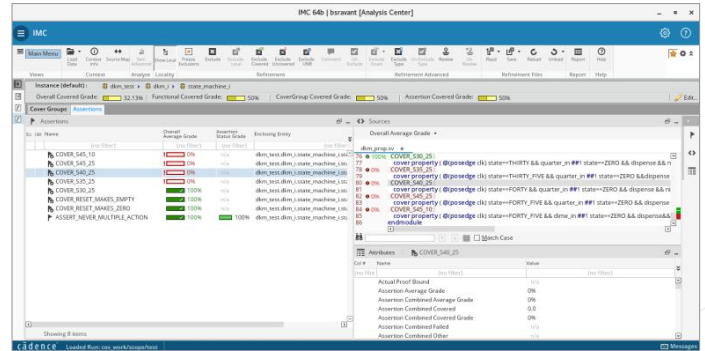
What Is Control-Oriented Functional Coverage?



Control-oriented functional coverage is functional coverage that focuses on *control* signals. It measures sequences of control signal states.

Control-oriented functional coverage measures:

- Completion of covered user-defined Boolean signal sequences
- Completion of asserted user-defined design properties (for example, arbitration)
- Additional control coverage that the tool can infer from assertions



Abstraction level	Signal
Collected	Asynchronously/Synchronously
Domain	Control

9 © Cadence Design Systems, Inc. All rights reserved.

cadence

Control-oriented functional coverage focuses on control signals:

- What control conditions occurred;
- What control sequences occurred; and
- What interactions between control sequences occurred.

Control-oriented functional coverage measures control signal sequences, specified either:

- Explicitly by the user with **cover** directives;
- Implicitly by the user with **assert** directives; or
- Inferred by the tool from instantiated assertion IP.

So control coverage is a natural extension of assertion-based verification.

Control-oriented functional coverage is collected at the signal level of abstraction, asynchronously or synchronously, in the control domain.

Control-oriented functional coverage is:

- More difficult to specify than code coverage, as the user defines the conditions and sequences; but
- Less difficult to interpret than code coverage, as it directly identifies the failing condition or sequence.

The image is of assertion analysis in the Integrated Metrics Center. We have selected an uncovered “cover” directive. The Source pane displays the source of that directive. It is the sequence that in any cycle the coin count is FORTY and the signal “quarter in” is true, and in the next cycle, the coin count resets to ZERO. We covered this sequence as a property, which for this design, produces the same result as covering it as a sequence.

What Is Transaction Coverage?



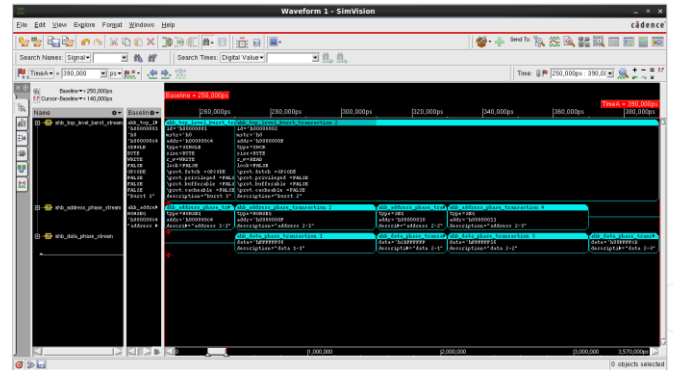
A *transaction* is an abstraction of exchanged information, and also the act of exchanging that abstraction. A transaction is implemented as a structure of numerical attributes, for example, *control* attributes and *payload* attributes.

Transaction coverage is implemented as functional data-oriented coverage of those attribute values, sampled upon transaction transmission, and potentially also at waypoints and at reception.

Transaction coverage measures:

- Explicitly at transaction level
 - Transaction types and payloads
- Implicitly at signal level
 - Transaction forms (implied control)

Abstraction level	Transaction	Signal
Collected	Asynchronously	Synchronously
Domain	Data	Control/Data



10 © Cadence Design Systems, Inc. All rights reserved.

cadence

Transaction coverage focuses on transaction attributes:

- What transaction types (control attributes) occurred; and
- What transaction payload attributes occurred.

Transaction coverage, at the transaction level of abstraction, explicitly measures transaction types and payloads, in terms of attribute values. An abstract transaction, has control attributes, that indicate what type of transaction it is, for example, a read transaction, or a burst-write transaction, and a payload attribute, that is, the data conveyed as payload by the transaction.

Transaction coverage, at the signal level of abstraction, implicitly measures transaction types, in terms of control signal sequences. Transactions consist of data values, packaged in an envelope of control signals that mediate the data transfer, so at the signal level, you can consider them to involve both control and data coverage. The transaction type, implies the sequence of control signals required to send the transaction, so abstractly represents the control signals. Here, we use the “form” word to represent the signal-level control and data coverage implied by transaction transmission and reception.

Transaction coverage is collected asynchronously at the transaction level of abstraction, and synchronously at the signal level of abstraction.

Quiz: Choosing Coverage Types



For each scenario, choose the most appropriate coverage type and state whether you would collect it asynchronously or synchronously (or either).

- Basic coverage of RTL design code
- Coverage of FSM interactions
- Coverage of bus protocols
- Thorough coverage of FSM states
- Coverage of inter-unit transactions



For each scenario, choose the most appropriate coverage type, and state whether you collect it asynchronously, synchronously, or either.

Implementing Coverage



Coverage is implemented to measure how well the testbench verifies the design.

Implementing coverage uses the four basic steps introduced here.

- 1 Instrument the design
- 2 Collect the coverage data
- 3 Reduce the coverage data
- 4 Analyze the coverage data



Implementing coverage is in the four basic steps introduced here.

- In Step 1, the tools automatically add coverage monitors, to explicit or implicit coverage points, and the user manually adds coverage monitors, for functional coverage.
- In Step 2, the monitors count the coverage point hits throughout the simulation.
- In Step 3, the tools reduce the coverage data, to make it more manageable, and record the data to coverage databases, for later analysis.
- In Step 4, the user reports or displays the coverage data, and analyzes the result, to determine additional test requirements.

Step 1: Instrument the Design

Instrument the design:

- **Automatically** (done by tool)
 - Monitors to track HDL code execution (block, branch, expression...)
 - Monitors to track FSM states and transitions
 - Monitors to track node toggles
- **Semi-Automatically** (implied by user)
 - Monitors implied by user-written pragmas
 - Monitors implied by PSL or SVA assertions
 - Example: Instances of IAL / OVL assertions
- **Manually** (done by user)
 - PSL or SVA cover directives
 - SystemVerilog covergroups

1	Instrument the design
2	Collect the coverage data
3	Reduce the coverage data
4	Analyze the coverage data



The 1st step in the coverage process is to instrument the design.

- The tool would ideally automatically instrument the design for code coverage, requiring little user interaction.
- The tool may infer coverage for user-written pragmas and assertions.
- For functional coverage, the user must write **cover** directives and SystemVerilog covergroups, requiring detailed user knowledge of the design specification.

NOTE:

Incisive Assertion Library (IAL): The Incisive Assertion Library (IAL) is collection of reusable verification components containing

predefined assertions and coverage points. The IAL components (IAL-Cs) are designed for the functional verification of commonly used hardware structures.

Open Verification Library (OVL): The Open Verification library of assertion checkers is intended to be used by design, integration, and verification engineers to check for good/bad behavior in simulation, emulation, and formal verification.

Step 2: Collect the Coverage Data

Collect coverage data in accordance with several criteria:

- In what domain { data, control }
 - For example, transactions are explicitly data but also imply control signal sequences
- At what abstraction level {code, signal, transaction}
 - For example, transactions are explicitly transaction level but also imply signal level
- How collected { asynchronously, synchronously }
 - For example, you can sample control/data states asynchronously or synchronously

1	Instrument the design
2	Collect the coverage data
3	Reduce the coverage data
4	Analyze the coverage data



The 2nd step in the coverage process is to collect coverage data.

You can categorize coverage in accordance with several criteria:

- You can categorize coverage by whether it is data-oriented or control-oriented.
- You can categorize coverage by the level of abstraction at which it is collected.
 - Code level, that is, what is going on in the source code. Code-level behavior is generally asynchronous, that is, above the level at which clocks are defined.
 - Signal level, that is, what inter-element communication is occurring in the net list. Signal-level behavior can be either asynchronous or synchronous.
 - Transaction level, that is, what inter-block communication is occurring at the system level. Transaction-level behavior is asynchronous at the transaction level, and synchronous at the signal level.
- You can categorize coverage by how it is collected, asynchronously or synchronously.

Step 3: Reduce the Coverage Data

Reduce coverage data by:

- For signal-level data:
 - Binning
 - Counting the occurrences of values within each of several specific ranges
 - Crossing
 - Counting the co-occurrences (cross-products) of related behaviors
- For transaction-level data:
 - Recording
 - Transaction type as behavior; attributes as data values.
 - Querying
 - Transaction history, for types of transactions and their relationships.

1	Instrument the design
2	Collect the coverage data
3	Reduce the coverage data
4	Analyze the coverage data



The 3rd step in the coverage process is to reduce the coverage data.

You could quickly become overwhelmed by the volume of signal-level data.

You might want to reduce the data to a form more easily analyzed.

For the synchronous, signal-level coverage types, coverage information consists of counts of the number of occurrences of the specified behaviors. You can reduce these counts by grouping, that is, binning them together, and further reduce them by considering the co-occurrences of related behaviors, that is, cross-products, or crossing.

For transaction-level coverage, the coverage information consists of the times at which the transaction starts and ends, and the transactions themselves, including the signal-level behavior that structures each transaction, and the data associated with the transaction. This information is recorded in an abstract form, where the transaction type represents the precise signal-level behavior that frames the transaction, and the transaction attributes represent the data values associated with the transaction. You can query the transaction history, to determine the types of transactions that occurred, and their relationships.

Step 4: Analyze the Coverage Data

Analyzing the coverage data (interactively or by scripts) can include:

- Database operations (merge)
 - Merge – to merge data from multiple coverage databases from potentially different design hierarchies to a new database
 - Also sometimes available:
 - union – To merge data from multiple independent runs
 - intersection – To cull duplication from the test suite
 - difference – To validate recent test enhancements
- Refinement
 - To exclude types, instances, metrics, or items that cannot be covered
- Report generation (text or HTML)
 - To track progress and status of verification efforts
 - To steer verification effort to less covered design areas
- Test ordering
 - To determine the relative value of tests within the test suite

- 1 Instrument the design
- 2 Collect the coverage data
- 3 Reduce the coverage data
- 4 Analyze the coverage data



The 4th step in the coverage process is to analyze the coverage data.

You can analyze the collected coverage data interactively, by the textual or graphical user interfaces, or non-interactively, by driving the textual interface with a script.

You can:

- Merge the coverage data of multiple tests
- Exclude items that cannot or should not be tested
- Generate reports in multiple formats
- Order tests so that your test suite executes the most effective tests first

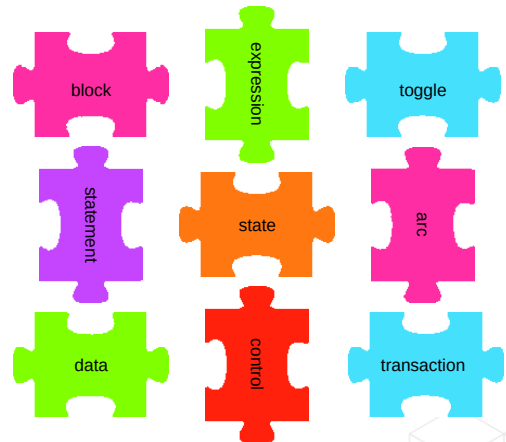
What Is a Comprehensive Coverage?



A **Comprehensive** coverage solution would offer code coverage, FSM coverage, data-oriented and control-oriented functional coverage, and some form of transaction coverage.

Comprehensive coverage would include:

- Code coverage
 - Analyze HDL code structure – which blocks of design code executed
 - Determine how fully *code structure* is exercised
- Functional coverage
 - Analyze design functionality – which functions such as Add and Queue Filled executed
 - Determine how fully *design behavior* is exercised
- Transaction coverage
 - Analyze meta-level transactional behavior of the design – which high-level transactions such as Bus Read or Send Complete occurred
 - Determine how fully *block-level interaction* is exercised



17

© Cadence Design Systems, Inc. All rights reserved.

cadence

Comprehensive coverage would include code coverage, functional coverage and transaction coverage.

Code coverage measures how completely the test exercises design code, for example, what code blocks executed, what expression terms controlled the expression outcome, and what signals toggled. Tools can automatically instrument the code for coverage, but the tools do not understand the code purpose.

Functional coverage measures how completely the test exercises design functionality. As tools do not understand the code purpose, you need to yourself define the functionality to cover.

Transaction coverage is at an even higher level of abstraction. It records the occurrence and sequence of design behavior, as represented by transactions.

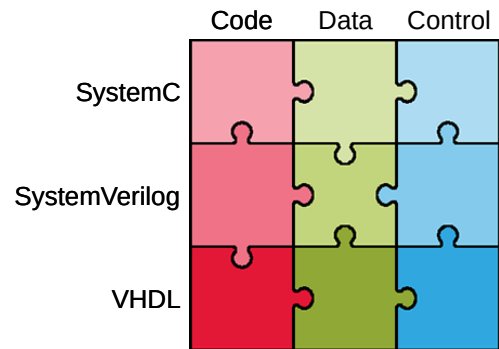
What Is Xcelium Integrated Coverage?



The **Xcelium Integrated Coverage** is a single infrastructure, with a single use model, high performance, and comprehensive coverage capabilities, that are integrated with verification management methodologies.

The Xcelium Integrated Coverage offers:

- One infrastructure (simulation and analysis environment) for all coverage
- One simple, consistent use model for all coverage types
- A high performance, native code implementation
 - No separate source code instrumentation step!
- *Comprehensive* coverage capabilities:
 - Integrated block, branch, expression, toggle, FSM coverage
 - Data-oriented functional coverage using SystemVerilog covergroups
 - Control-oriented functional coverage using PSL and SVA
- Integration with verification management testbench automation



18

© Cadence Design Systems, Inc. All rights reserved.

cadence

The Xcelium Integrated Coverage is a single infrastructure, with a single use model, high performance, and comprehensive coverage capabilities, that are integrated with verification management methodologies.

The Xcelium Integrated Coverage offers:

- One infrastructure for all coverage
- One simple, consistent use model for all coverage types
- A high performance, native code implementation
- Comprehensive coverage capabilities
- Integration with verification management testbench automation

The Xcelium Integrated Coverage implements the Accellera™ Systems Initiative “Unified Coverage Interoperability Standard (UCIS)”. This standard does not directly support recording transactions. You instead record the sampled values of pertinent transaction attributes.

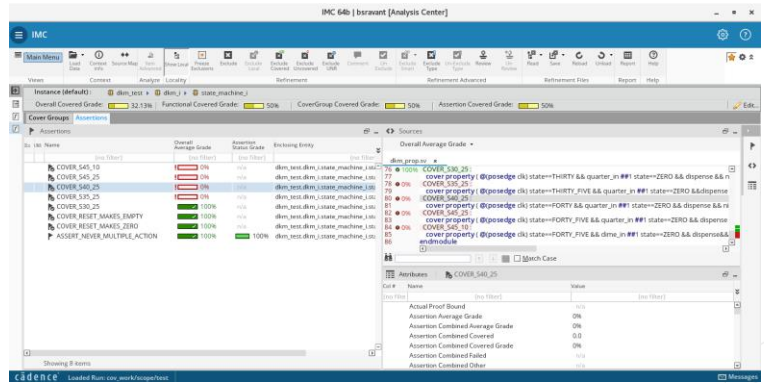
What Is Code Coverage in Xcelium Simulation?



For Xcelium™ simulation, code coverage is that coverage that the elaborator can instrument by analyzing the source code.

Code coverage includes:

- Block coverage
- Expression coverage
- Toggle coverage
- FSM coverage



19 © Cadence Design Systems, Inc. All rights reserved.

cadence

The Xcelium™ Integrated Coverage defines “code coverage” as that structural coverage that the tool can instrument without your functional specification. That definition includes block coverage, expression coverage, toggle coverage, and FSM coverage.

The image is of assertion analysis in the Integrated Metrics Center. We have selected an uncovered “cover” directive. The Source pane displays the source of that directive. It is the sequence that in any cycle the coin count is FORTY and the signal “quarter in” is true, and in the next cycle, the coin count resets to ZERO. We covered this sequence as a property, which for this design, produces the same result as covering it as a sequence.

What Is Block Coverage?



Block Coverage is a basic code coverage mechanism that identifies which blocks in the code have been executed and which have not.

What is a block?

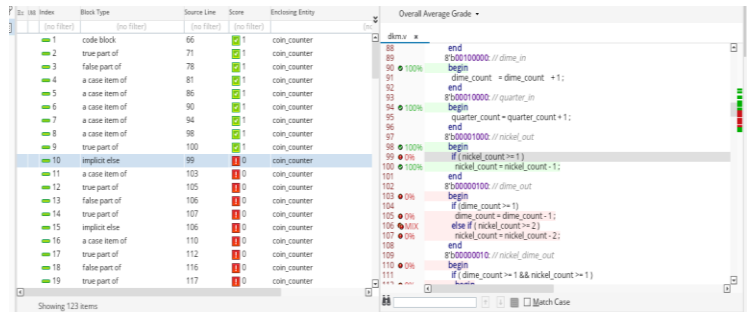
- Normally a set of statements (may be only one) that always execute together – either none execute or all execute.

What does block coverage do?

- Monitor and count block entry.
- Calculate and report block entry statistics.

Why perform block coverage?

- To measure how completely your testbench exercises the design code.
- The most effective (thus usually first) code coverage type employed.



Block coverage is a basic code coverage mechanism that identifies which blocks in the code have been executed and which have not.

A block is a contiguous set of statements that always execute together. Either none of the statements execute, or all of the statements execute. Any statement that breaks the execution flow, starts another block.

Block coverage identifies whether test scenarios exercise the statements in a block. It monitors and counts block entry, and then during analysis, calculates and reports block entry statistics.

Block coverage is the simplest and easiest-to-visualize metric for measuring code coverage. For effective test cases, block coverage should ideally be near 100%. However, even 100% block coverage does not necessarily imply verification completeness. In general, block coverage is an essential first step in the overall verification process.

The image depicts block analysis in the Integrated Metrics Center. For the selected block, the Source pane highlights that block. Here, you see that one unexecuted block started on line 99.

Quick Reference Guide: Flow Break Constructs



Flow Break Construct	SystemVerilog
begin (explicit or implicit)	begin, fork
branch statements	if, else, case , case item
loop statements	for, forever, repeat, while, disable, continue, break
event and timing control	wait, @, #
subroutine execution	function/task call and return



A block is a contiguous set of statements that always execute together. Any construct that breaks execution flow creates a new block.

What is normally scored is:

- For SystemVerilog, code lines within a function, task, or procedural block
- For VHDL, code lines within a function, procedure, or process

Example: Flow Break Constructs and Code Blocks

```
// SystemVerilog
begin                // BLOCK
    statement_or_null
end

if (a==b)
    statement_or_null // BLOCK
else
    statement_or_null // BLOCK

case (x)
    0 :
        statement_or_null // BLOCK
    default :
        statement_or_null // BLOCK
endcase

while (a==b)
    statement        // BLOCK

# 10
statement            // BLOCK
```

```
// SystemVerilog
10 always_ff
11 ☒ begin
12 ☒ @ ( posedge clk );
13     variable1 <= value1;
14     if ( condition1 )
15 ☒     variable2 <= value2;
16     else
17 ☒     begin
18         variable2 <= value2;
19         variable4 <= value4;
20     end
21 ☒ variable5 <= value5;
22     if ( condition2 )
23 ☒     variable6 <= value6;
24 ☒     // implicit else
25 ☒     variable7 <= value7;
26     end

// 8 blocks
```

22 © Cadence Design Systems, Inc. All rights reserved.



Any construct that breaks execution flow creates a new block. This illustrates for some typical SystemVerilog and VHDL code, where the tool considers a new block to begin.

Here are the three rules for what is a block:

- All statements between the matching keywords “**begin**” and “**end**”, and do not contain a flow break statement, or a single statement that *could* have the keywords “**begin**” and “**end**” added around it, are a block.
- All statements from the keyword “**begin**”, up to and including completion of the next flow break statement, are a block.
- All statements from *after* completion of a flow break statement, up to and including the next flow break statement, or until the final keyword “**end**”, are a block.

What Is Branch Coverage?



Branch coverage augments block coverage to also record execution of code statement branches.

What is a branch?

- A statement that conditionally executes.

What does branch coverage do?

- Monitor and count branch execution.
- Calculate and report branch execution statistics.

Why perform branch coverage?

- To more precisely measure the execution of individual code branches.
- The only way to measure execution of code branches that are not also blocks.

Most branches are scored as blocks, thus branch coverage may overweight the coverage metrics.

Here are some examples of branches that are by default *not* scored as blocks.

```
// SystemVerilog continuous assignment

assign y = c ?
           a : // Branch
           b ; // Branch
```

```
-- VHDL concurrent assignment

y <=
    a when c = '1' -- Branch
  else b ;         -- Branch

with c select z <=
  a when '1',      -- Branch
  b when others ; -- Branch
```

Branch coverage augments block coverage to also record execution of code statement branches.

A branch is a statement that conditionally executes, for example, the branches of an “**if**” or “**case**” statement, conditional operator expressions, and VHDL selected signal assignments.

Branch coverage separately monitors and counts branch execution, and then during analysis, calculates and reports branch execution statistics.

You perform branch coverage to more precisely measure the execution of individual code branches, and because it is the only way to measure execution of code branches that are not by default scored as blocks, for example, conditional operator expressions and VHDL concurrent assignments.

Branch coverage separately covers the branches of “**if**” and “**case**” statements. Each case item statement is already scored as a block, but branch coverage individually scores the multiple “**case**” branches that execute the same one case item statement.

Branch coverage covers the branches of the Verilog ternary assignment, both as a procedural assignment and as a continuous assignment.

Branch coverage covers the branches of the VHDL conditional and selected signal assignments, both as a sequential assignment and as a concurrent assignment.

The configuration command “**set_assign_scoring**” by itself scores the Verilog continuous assignment as one block.

The configuration command “**set_branch_scoring**” by itself scores *branches* both as blocks and as branches.

Example: Code Branches

```
// SystemVerilog

assign y =                // Block
  (i==4'b1000) ?
    2'b11 :                // Branch
  (i==4'b0100) ?          // Branch
    2'b10 :                // Branch
  (i==4'b0010) ?          // Branch
    2'b01 :                // Branch
    2'b00 ;                // Branch

// 0: by default
// 1: set_assign_scoring
// 6: set_branch_scoring
```

24 © Cadence Design Systems, Inc. All rights reserved.



The Xcelium™ Integrated Coverage by default omits to score SystemVerilog continuous assignments. To score SystemVerilog continuous assignments as blocks, use the coverage configuration command “set assign scoring” (**set_assign_scoring**). That configuration command scores each continuous assignment as one block. To score the conditional expression individual branches, use the coverage configuration command “set branch scoring” (**set_branch_scoring**).

The Xcelium Integrated Coverage omits to score VHDL concurrent signal assignment statements. The coverage configuration command “set assign scoring” does not apply to VHDL. To score the concurrent signal assignment individual branches, use the coverage configuration command “set branch scoring”.

The command “set branch scoring” enables branch scoring throughout the design, in both procedural and non-procedural assignments.

Note that the simulator evaluates all non-procedural assignments during the simulation initialization phase.

For the SystemVerilog example, the tool by default does not score the continuous assignment, with the command “set assign scoring”, scores it as one block, and with the command “set branch scoring”, individually scores the true and false parts of each conditional expression.

For the VHDL example, the tool by default does not score the concurrent assignment, and with the command “set branch scoring”, individually scores each selection.

The configuration command “**set_assign_scoring**” by itself scores the SystemVerilog continuous assignment as one block.

The configuration command “**set_branch_scoring**” by itself scores *branches* both as blocks and as branches.

What Is Expression Coverage?



Expression Coverage is a mechanism that factors logical expressions and monitors them during the simulation run.

What is an expression?

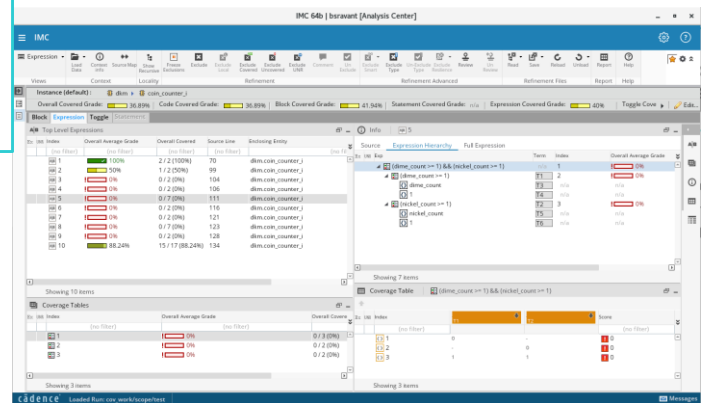
- A conditional expression of logical operators & scalar terms
 - You can optionally also cover other statements
 - You can optionally also cover other operators

What does expression coverage do?

- Monitor and count operand states
- Calculate and report term value coverage

Why perform expression coverage?

- A valuable next step after high block coverage
- Helps explain why a block executed



Expression coverage involves a multitude of dimensions:

- Which statements and operators to include?
- How to score the expressions?

Expression Coverage is a mechanism that factors logical expressions and monitors them during the simulation run.

The Xcelium Integrated Coverage scores expression coverage for by default only conditional expressions of scalar terms and binary logical operators (Verilog: `||`, `&&`; VHDL: `OR`, `AND`, `NOR`, `NAND`).

- You can optionally configure the tool to also score other statements, for example, assignment statements.
- You can optionally configure the tool to also score other operators, for example, relational operators.

Expression coverage measures how completely the testbench exercises expressions, such as in assignments and procedural branching constructs (**if/case** conditions).

Expression coverage is the most natural next verification step after you attain high block coverage. It goes beyond block coverage to help explain *why* the execution thread entered a block.

Expression coverage involves a number of dimensions, for example:

- Which statements and operators to include; and
- How to score the expressions.

The image depicts expression analysis in the Integrated Metrics Center. For the selected expression, the Information Tabs pane Expression Hierarchy tab displays the expression hierarchy, and the Coverage Table pane displays the coverage table. Here, you see that no row of that table was hit. That is, none of the sum-of-products input combinations occurred.

What Is Toggle Coverage?



Toggle Coverage is a basic netlist coverage mechanism that identifies which nets in the design have toggled and which have not.

What is a signal toggle?

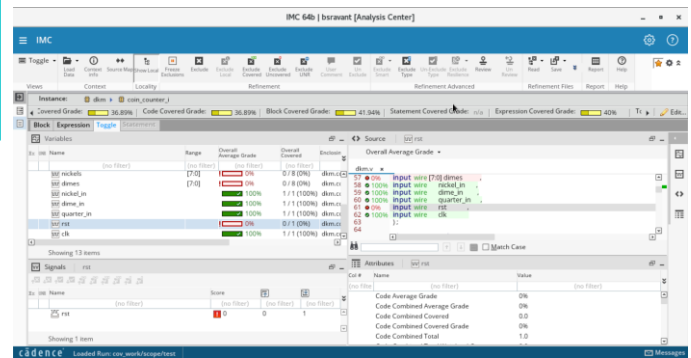
- A binary transition, and return after a finite delay, of a DUT signal
- Signals may transition through, but not terminate at, an unknown state

What does toggle coverage do?

- Monitor and count signal toggle activity
- Calculate and report signal toggle coverage

Why perform toggle coverage?

- It is the only “code” coverage available for a gate-level netlist
- It verifies that design interconnect is connected and “wiggles”



Toggle Coverage is a basic netlist coverage mechanism that identifies which nets in the design have toggled and which have not.

The simulator normally records the transitions 0-to-1 and 1-to-0, and the reporting tool reports as full toggles the transitions 0-to-1-to-0 and 1-to-0-to-1, and reports other binary transitions as partial toggles.

Toggle coverage monitors, collects, and reports signal toggle activity. Toggle coverage does not apply to integer or real types. You can also optionally exclude module ports. The Xcelium™ Integrated Coverage collects and reports signal activity in the VHDL and Verilog portions of the design.

Toggle coverage is difficult to interpret, but is the only code coverage available for a gate-level net list. It verifies that the design interconnect is driven and exercised.

The image depicts toggle analysis in the Integrated Metrics Center. For the selected variable, the Source pane shows its declaration, and the Signals pane shows for each bit, which, if any, direction that it toggled. Here, you see that the reset signal toggled down but not up.

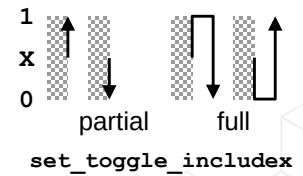
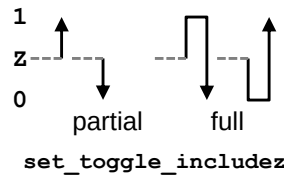
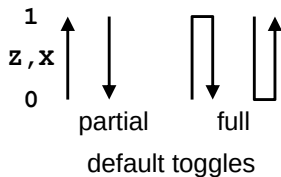
What Is a Toggle?



A toggle is a binary transition, and return after a finite delay, of a DUT signal. Signals may transition through, but not dwell at, an unknown state.

With **set_toggle_includez** toggles can originate in (but not end in) high impedance (Z).

With **set_toggle_includex** toggles can originate in (but not end in) unknown state (X).



27 © Cadence Design Systems, Inc. All rights reserved.

cadence

The simulator normally records the transitions 0-to-1 and 1-to-0, and the reporting tool reports as full toggles the transitions 0-to-1-to-0 and 1-to-0-to-1, and reports other binary transitions as partial toggles.

To record also the transitions originating in, but not ending in, the high-impedance state, include the coverage configuration command “set toggle include z” (**set_toggle_includez**).

To record also the transitions originating in, but not ending in, the unknown state, include the coverage configuration command “set toggle include x” (**set_toggle_includex**).

For the transition to be recorded, the new signal state must persist for at least the strobe interval. The strobe interval is by default the simulation time precision. To filter wider glitches, set a wider strobe interval with the coverage configuration command “set-toggle-strobe” (**set_toggle_strobe**).

Quick Reference Guide: Toggle Constructs Covered By Default



	SystemVerilog
Scopes	Interface and module ports, nets, variables.
	Not inside generate .
Types	Nets of scalar types tri , triand , trior , triereg , tri0 , tri1 , wire , wand , wor . Variables of scalar types bit , logic , reg .
	Nets of single-dimension packed vectors of types tri , triand , trior , triereg , tri0 , tri1 , wire , wand , wor . Variables of multi-dimension vectors of types bit , logic , reg .
	Members of above types in non-arrayed structs.

To cover SystemVerilog type-parameterized nets and variables use the CCF command
`set_parameterized_module_coverage { -type_parameter_only | -all_parameter }`



The Xcelium™ Integrated Coverage, covers by default SystemVerilog interface and module ports, nets, and variables not in a generate construct, of scalar and packed vector nets of the listed types, and scalar and vector variables of the listed types, and non-array structures of these types.

The Xcelium Integrated Coverage, covers by default VHDL entity ports and architecture signals not in a generate construct, of scalar BIT, BOOLEAN, and logic, and vector BIT and logic, and non-array records of these types.

What Is FSM Coverage?



FSM Coverage identifies design FSMs and covers their states and transitions, and optionally also arcs.

What is a FSM?

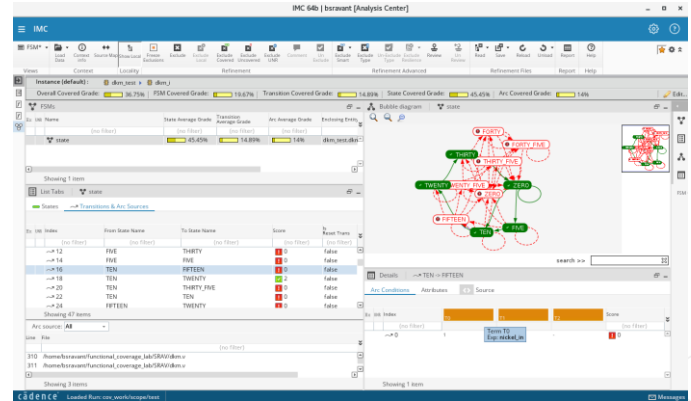
- A FSM consists of sequential logic to hold the current design state and combinational logic to compute the next design state.

What does FSM coverage do?

- Monitor and count which FSM states occur, which transitions between FSM states occur, and optionally why the transitions occur
- Calculate and report FSM coverage

Why perform FSM coverage?

- FSM coverage is more closely related to design functionality but is still automatically instrumented similarly to code coverage.



FSM Coverage identifies design state machines and covers their states and transitions, and optionally also their arcs.

A state machine consists of sequential logic to hold the current design state and combinational logic to compute the next design state. You can visualize a state machine as a state diagram depicting a finite number of states and the possible transitions between those states.

FSM coverage monitors and counts which state machine states occur, which transitions between state machine states occur, and optionally why the transitions occur, and calculates and reports the FSM coverage.

FSM coverage raises the abstraction level at which you analyze coverage. While still being automatically instrumented as code coverage, you can more easily relate coverage report items to actual design functionality.

The image depicts FSM analysis in the Integrated Metrics Center. We have selected the FSM “state” and the transition TEN to FIFTEEN. The Bubble Diagram highlights the selected state or transition. The Details pane Arc Conditions tab displays the conditions enabling each arc. Here, you see one arc between state TEN and state FIFTEEN, and its condition is that the signal “nickel in” is true.

What Is a Coverable FSM?



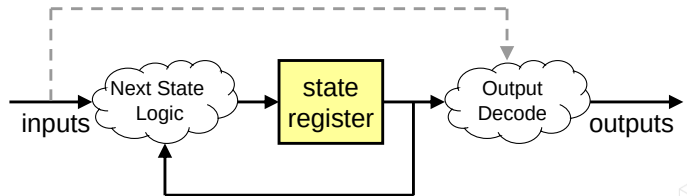
A Coverable FSM is an FSM that the tool can statically extract.

Code guidelines for FSM extraction:

- Clock is a single edge of a single scalar signal
- Each reset (any number) is a single scalar signal
 - Can be synchronous or asynchronous
- State is a single whole vector
 - Assigned only constant values, not variables
 - Exception – you may utilize the documented rigorous Verilog one-hot coding style

The tool can still extract some Verilog FSMs that violate these guidelines

- See documentation.



A Coverable state machine is an state machine that the tool can statically extract.

Code within these guidelines to facilitate state machine extraction:

- For the clock, use a single edge of a single scalar signal.
- For optional resets, use scalar signals, or single bits of a vector signal.
- Assign constants to the state vector, and if you have one, the next state vector, as a whole, and not as a bit or part select, except that when adhering to a strict one-hot coding style, you may make bit assignments to the state vector.
 - You can define state values by using enumerated types or VHDL generics and Verilog parameters.
 - The extraction engine will ignore the assignment of a variable to the state vector.

The tool can extract *some* Verilog state machines that violate these guidelines, for which you should refer to the Cadence® document “Integrated Coverage User Guide”.

Module Summary

In this training module, you

- Identified the coverage types to apply to your project

This training module discussed:

- Coverage fundamentals
 - Determine your coverage goals
 - Select coverage types
- Coverage basic process
 - Instrument the design
 - Collect the coverage data
 - Reduce the coverage data
 - Analyze the coverage data
- What the Xcelium Integrated Coverage offers

31 © Cadence Design Systems, Inc. All rights reserved.



You should now be able to attain your objective, which is to identify where coverage fits in your design flow.

The goal of coverage is to measure how completely the testbench verifies the design:

- To identify design areas in which to focus verification efforts; and
- To estimate the remaining verification effort.

Coverage types include:

- Code coverage, including blocks, branches, and expressions;
- Toggle coverage;
- FSM coverage, including states, transitions, and arcs;
- Functional coverage, which can be data oriented or control oriented; and
- Transaction coverage.

The coverage process is this:

- 1st – to instrument the design. Tools automatically add coverage monitors to explicit and inferred coverage points, and the user adds functional coverage monitors such as SystemVerilog covergroups.
- 2nd – to collect the coverage data. Coverage point monitors count occurrences of coverage point hits during the simulation.
- 3rd – to reduce the coverage data. Tools bin and cross coverage data to make it more manageable, and record the data to coverage databases for later analysis.
- 4th – to analyze the coverage data. The reporting tool can merge coverage from multiple runs and report coverage statistics using user-selected formats. The user then diagnoses the reports to determine the additional test requirements.

The Xcelium Integrated Coverage is a single infrastructure, with a single use model, high performance, and comprehensive coverage capabilities, that are integrated with verification management methodologies.